《冯诺依曼计算机结构》读书报告

2200013146 郭劲豪

一、报告摘要

第一章讲述了针对超高速自动数字计算系统结构的一些思考，以及对于自动计算系统结构的名词解释，阐述了自动计算系统的定义、其运行的输入需求，和自动这一词（即不需要人为干预）的解释。

第二章通过类比人类神经元活动和计算机进行运算过程，介绍了自动数字计算系统中的 5 个特定部件：中央算术部件(CA)、中央控制器(CC)、存储设备(M)和输入输出设备(I/O)，同时还介绍了外部记录介质(R)。

第三章简要说明如何对这 5 种部件进行设计，在设计不能够按照一定顺序一个一个完成设计时，需要合理调配设计步骤来实现机器结构的实现。

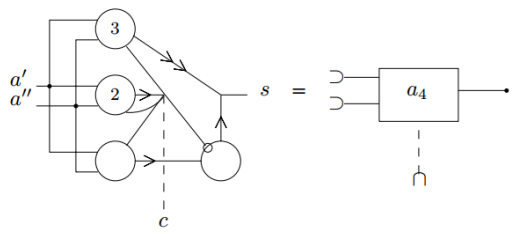
第四章介绍了电子元件所需要的两种状态（平衡态和激发态），提到了设备中非常重要的一环：时钟，来确定电子设备的时序逻辑。在此基础上作出假设，考虑采用真空管元件实现精准的突出延迟。

第五章开始阐述中央算术部件(CA)的运算原理，我们通过电子元件所处的两种状态很自然地联想到使用二进制来做运算。在非真空管元件中我们或许要通过一些技巧（缩放操作）来减少运算所消耗的时间，但是在真空管元件中，我们只需要考虑让过程尽可能地简单，设备尽可能地包含少的原件。在这系列的思考过程中，也认识到了完全相反观点的启发意义。

第六章引入了 E-元件的定义，将它和神经元进行类比，我们就可以把电路设计类比于人类神经元的构建，有兴奋性和抑制性的不同种类，还能够对各种刺激做出复杂的反应。在这里我们约定，不考虑回溯的刺激，也就是说

第七章具体描述了加法和乘法的电路构造。我们考虑设计一个如 Figure3 的加法器，

FIGURE 3



因为加法我们每一位数字只使用一次，所以不需要存储很长的时间，考虑用两个输入线读入加数，同时用一个输出线输出结果，用另一个输出线输出进位即可。注意在进行加法运算的时候要保持时序一致，同时进入下一个时序（即下一位的加法）。

但是到了考虑乘法的时候，我们每一个乘数的位都需要乘若干次，假设有 30 位则需要乘 30*30=900 个单位时间次。所以我们需要把某一位保存很长的时间。考虑建立一个一种自我刺激的元件作为存储设备，然后再这个基础上，建立一个乘法器网络：即将 Figure7 和 Figure8 结合。

FIGURE 7

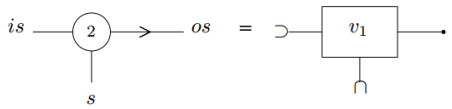
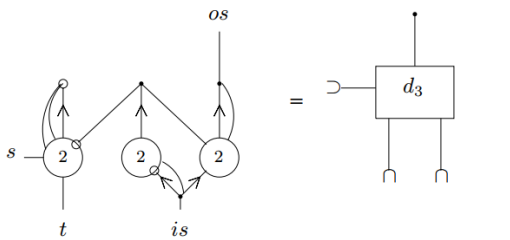
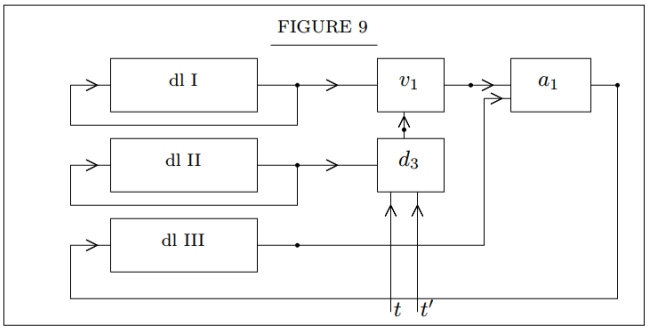


FIGURE 8



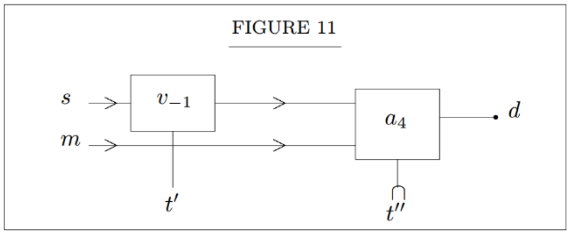
也就是说将被乘数移动一定位数，然后通过 Figure8 的判断网络来决定是否叠加到答案

上。最后就像 Figure9 所展示的一样。



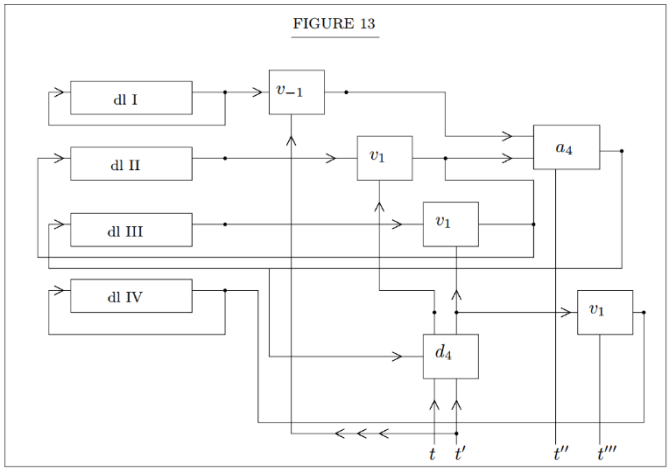
也就是说，将被乘数存储在 dlI 中，乘数存储在 dlII 中，最后的结果存储在 dlIII 中，通过上述所说明的线性延迟设计来保证 t,t’的时序。但是这样子的设计也有问题，我们仍然不知道如何处理符号问题，以及二进制小数应该如何处理。

第八章则是讨论的减法和除法网络的实现。现在我们考虑将一个二进制数引入符号位和小数点，最左边的一位表示当前这个二进制数的符号（如果是 1 那么就是负的，否则就是正的）。在此基础上，我们建立一个减法网络，如 Figure11 所示。我们提前用一个元件来判断当前这个是不是负数即可。剩下的则是和加法网络同理。接下来我们考虑除法，和乘法基本同理，我们也引入一个新的判别器来判断除数当前为是不是 1。这样子的延迟就来到了 3t。



我们考虑如 Figure13 的设计：

将被除数在 dl I 中循环，除数存储在 dl II 中，同时将原始数据存储在 dl III 中，最终的结果我们放在 dl IV 里面，剩下的只需要按照若干次正负数判断和减法即可。



第九章简单的介绍了一些二进制的规则，比如说小数数应该如何用二进制表示，以及简单的舍入规则，以及其对应的元件电路设计（参见原文 Figure14）

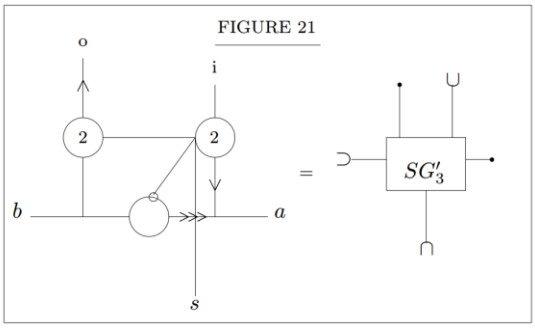
第十章则是介绍了如何构建根号的网络，大致与除法的网络类似（dl I 将不再被使用），然后则是讨论了 CA 中是否值得（或者说有必要）包括这些基本的运算。笔者认为加法和减法作为最基本的运算是必须的，而乘法除法以及开根号这些运算是否必须则是存疑的。这样子会降低整个系统的精度，而且由于乘法除法开根号的运算周期显著地高，会导致计算机的吞吐量降低，影响计算机性能。第二个问题则是一些更加复杂的运算操作是否需要在 CA 中被包括，而这实际上取决于计算机的用途，这是不能够一概而论的。

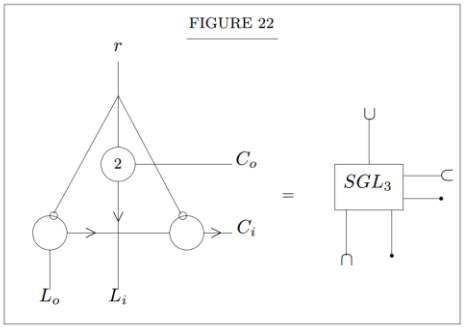
第十一章则是在讨论上述运算的实现都需要与 I/O 元件进行交互，我们应该如何实现 CA 和 M 之间的数据传输。读入的话，我们将每一个实数都读入 I_{ca} 中，而将第一个数在第二个数读入的时候转移到 J_{ca} 中（参见 Figure17）；与此同时，考虑通过 CA 将内存中的内容从一个部分转移到另一个部分，我们引入两个新的操作：i,j，表示将 I_{ca} 的数输出到

O_{ca}，以及将 J_{ca} 的输出到 O_{ca}。然后为了实现多于 2 个数字的输入，我们引入一个 s 运算，来根据 O_{ca} 的数值正负来决定后面两个数输入到 I_{ca} 和 J_{ca} 的顺序。与此同时，我们还应该实现十进制和二进制之间的互相转换。综上，CA 中需要实现+，-，×，÷，√，i，j，s，db，bd 这十种操作。

第十二章则是主要介绍了存储空间在二进制下的表示意义，以及将 32 个存储单元划分为一组，同是讨论在 2.4 中提到的关于存储的关键点(a)到(h)是如何实现的。存储最重要的则是它的容量，本章讨论了如何通过 dl 这个元件来实现 M 的存储容量，也能得到计算公式 $c = h \times k$ ，c 代表 total capacity，k 代表 capacity of a dl organ, h 代表 number of dl organs in M。最后叙述了关于如何更好的管理内存空间的解决方案。

第十三章则是具体阐述了如何实现 M 中的元件（即 SG 和 SGL’），A 实际上是一个仅仅是将出现的脉冲恢复到其最初进入时的形状和强度（在 12.5 中已经讨论过了），而 SG 的构造如下图所示：





有了这两个元件我们就能够实现 Memory。

第十四章则是主要介绍 CC 如何从 M 中读取指令，CC 收到的命令自然分为以下四类：(a) CC 指示 CA 执行其十项具体操作之一的命令；(b) 命令 CC 将从命令一地转移到另一地；(c)命令 CC 变更内存获取地址，以便从那里获得下一个命令；(d) 控制设备的输入和输出操作的命令。这一章主要是介绍(b)和(c)，然后分析了获取指令所需要的时间周期。

第十五章则是结合前面章节讲述的 CC 所需要做的操作，把对应的操作进行了编码，使得计算机能够根据编码识别出对应需要做的程序和执行的操作。形如下图：

(I) Type	(II) Meaning	(III) Short Symbol	(IV) Code Symbol
			Minor cycle $l = (i_o)$ = $(i_0 i_1 i_2 \cdots i_{31})$
Standard Number or Order (γ)	Storage for the number defined by $\xi = i_{31} i_{30} \cdots i_1 = \sum_{i=1}^{31} i_i 2^{i-31}$ (mod 2) $-1 \le \xi < 1$. i_{31} is the sign: 0 for +, 1 for −. If CC is connected to this minor cycle, then it operates as an order, causing the transfer of ξ into I_{ca}. This does not apply however if this minor cycle follows immediately upon an order w → A or wh → A.	Nξ	$i_0 = 0$
Order (α) + (δ) Order (α) + (ϵ) Order (α) + (θ) Order (α)	Order to carry out the operation w in CA and to dispose of the result. w is from the list of 11.4. These are the operations of 11.4, with their current numbers w.decimal and w.binary, and their symbols w: w.decimal / w.binary / w: 0 / 0000 / +; 1 / 0001 / −; 2 / 0010 / ×; 3 / 0011 / ÷; 4 / 0100 / √; 5 / 0101 / i; 6 / 0110 / j; 7 / 0111 / s; 8 / 1000 / db; 9 / 1001 / bd h means that the result is to be held in O_{ca}. → $\mu\rho$ means that the result is to be transferred into the minor cycle ρ in the major cycle μ; → f, that it is to be transferred into the minor cycle immediately following upon the order ϵ; → A, that it is to be transferred into I_{ca}; no →, that no disposal is wanted (apart from h).	w → $\mu\rho$ or wh → $\mu\rho$ w → f or wh → f w → A or wh → A wh	$i_0 = 1$
Order (β)	Order to transfer the number in the minor cycle ρ in the major cycle μ into I_{ca}.	A ← $\mu\rho$	
Order (ζ)	Order to connect CC with the minor cycle ρ in the major cycle μ.	C ← $\mu\rho$	

综合上述十五章的描述，我们可以构造出一个简单的计算机模型，他是基于冯诺依曼结构而构建出来的，可以通过电子元件的刺激（也就是电压的变化），来完成一些基本的运算操作。

二、冯诺依曼结构特点分析

首先冯诺依曼结构最主要的特点就是：有 5 个最主要的组成结构构成，即 CC，CA，M，I 和 O。这样子的设计使得计算机的操作可以更加地专门化和功能化，每一个模块做模块对应所需要做的事情。

数据和程序均以二进制代码形式不加区别地存放在存储器中，具体而言就是无论是 CA 所需要的操作数，还是 CC 所需要读取的指令，都是存储在 Memory 空间内的，唯一的差别就是其是否含有标识位来区分指令和数据。

计算机在工作时能够自动地从存储器中取出指令加以执行：实际上也就是我们尽可能地降低人为的操控，让机器实现真正的自动。这也和存储程序这一设计息息相关，使得不需要手动重新连接电路，降低人为的干扰。

二进制表示也是重要的一环，报告采用了二进制系统来表示数据和指令，这使得计算机能够处理各种类型的信息，而且只需要用到两个状态（0/1），这也就让电子元件和神经元收到刺激的类比更加的自然、和谐。

处理器与存储器分离也是关键的一环，处理器（CC）和存储器（M）是分开的实体，它们之间通过总线进行通信。这种设计使得处理器和存储器可以独立地升级和扩展，提高了系统的灵活性。在如今，我们也是分开扩展内存/存储和 CPU 的。

三、当前计算设备分析对比

我们选取的是苹果公司在 2020 年末推出的 MacBook Air 作为对比设备，它采用的是当时苹果最为先进的 M1 芯片。虽然图中并未给出其内部的实际电路结构图、以及由于涉及到苹果公司的机密我们不可能得知，其设计的原理图纸。但是我们仍然可以从其参数和大部分现代化的处理器功能中将二者进行对比。

1. 存储容量和速度： M1 芯片内置了大量的高速缓存存储器，包括 L1、L2 和共享的 L3 缓存，这使得数据更快地可用，并提高了处理器的性能。相比之下，EDVAC 计划中的存储器容量非常有限，而且速度相对较慢。
2. 指令集和复杂度： M1 芯片采用了现代的 ARM 指令集架构（ISA），支持复杂且丰富的指令集，包括向量化指令、SIMD 指令、浮点运算指令等。与此相比，EDVAC 计划中的指令集相对简单，只包含基本的数据传输和算术运算指令。
3. 片上集成度： M1 芯片集成了多个功能和组件，包括 CPU、GPU、神经网络引擎（NPU）、安全芯片（Secure Enclave）等，使得整个系统更加紧凑高效。而 EDVAC 计划中的设计更为简单，只包含了基本的处理器和存储器以及和 I/O 设备简单的交互。
4. 并行处理能力： M1 芯片采用了多核心设计，支持并行处理和多线程操作，可以同时执行多个任务，提高了系统的整体性能和响应速度。相比之下，EDVAC 计划中的设计为单核心，不具备并行处理能力，单核到多核也是推动计算机性能提升的关键。从最初的单核到双核、四核、8 核，再到现在的 16 核，计算机的算力也在不断地提升。
5. 架构发展： 自 EDVAC 计划以来，计算机架构已经经历了多次演进和革新，现代处理器的设计考虑了许多先进的概念和技术，如动态执行、超标量处理、**分支预测**（ICS 课程中曾提到分支预测的精准度可以达到 55%）等，使得性能和效率得到了极大的提升。

四、“非冯结构”调研分析

常见的非“冯·诺依曼”结构的种类包括：

- 图灵完备性：这种结构包括图灵机以外的其他计算模型，如带记忆的有限状态机、递归函数、lambda 演算等。这些模型可能不符合冯诺依曼结构的存储程序模型，但仍然能够完成图灵完备的计算任务。
- 并行计算结构：并行计算结构包括多处理器系统（如多核 CPU、GPU）、集群系统、分布式系统等。这些系统中的处理器或计算单元可以同时执行多个任务，实现并行计算，与冯诺依曼结构相比，能够提高计算效率和性能。
- 量子计算结构：量子计算结构利用量子比特（qubit）而非经典比特来进行计算。量子计算机采用量子门和量子纠缠等特殊的运算方式，能够在某些情况下实现比经典计算机更高效的计算。
- 神经元结构：模仿生物神经元和神经网络的工作原理，采用类似于脉冲传递和神经突触连接的方式进行计算和信息处理。这种结构常用于人工神经网络和深度学习模型中。

这里主要简单阐述一下哈佛架构和量子计算机：

1. 哈佛架构：哈佛架构的微处理器通常具有较高的执行效率。其程序指令和数据指令分开组织和储存的，执行时可以预先读取下一条指令。这里就可以看出它和冯诺伊曼结构最大的差别，就是它的指令和数据是分开存储的，这也和我们现在计算机中用不同的 cache 来存储有着极大的相似性（因为就是从哈佛结构得到的灵感）。相比于冯诺依曼结构，优点显然是读取指令效率的提高，缺点也很明显，由于指令和数据并不存储在同一位置，所以需要两条总线来传输，一是提高了计算机的成本，二来也意味着制造需要更多时间。
2. 量子计算机：是一种使用量子逻辑进行通用计算的设备。与电子计算机（或称传统计算机）不同，量子计算用来存储数据的对象是量子比特，它用量子算法来操作数据。优点是：具有算力的显著提升，以及由于是量子运算，具有比较高的安全性，做到数据保护得更加完善。缺点则是技术不够成熟：目前量子计算机仍然处于研究和开发阶段，许多技术尚未完全实现，还需要不断的研究和改进；制造成本高：量子计算机的制造需要昂贵的设备和材料，导致制造成本非常高昂；以及由于技术尚未完善成熟导致的可靠性不足。

以上就是我关于阅读完 First-Draft-of-a-Report-on-the-EDVAC 后的读书报告，冯诺依曼结构是人类智慧的结晶，他让人类真正进入到了二进制——比特的时代，能够肆意地在二进制的海洋中遨游，在及短的时间内做到过去人类用自己的力量要做几百甚至几千年才能够做到的事情，这是一个伟大的发明。

参考文献列表

1. 维基百科，冯·诺伊曼结构，https://zh.wikipedia.org/wiki/%E5%86%AF%C2%B7%E8%AF%BA%E4%BC%8A%E6%9B%BC%E7%BB%93%E6%9E%84
2. 维基百科，Apple M1 芯片，https://zh.wikipedia.org/wiki/Apple_M1
3. 维基百科，哈佛架构，https://zh.wikipedia.org/wiki/%E5%93%88%E4%BD%9B%E7%BB%93%E6%9E%84
4. 维基百科，量子计算机，https://zh.wikipedia.org/wiki/%E9%87%8F%E5%AD%90%E8%AE%A1%E7%AE%97%E6%9C%BA
5. CSDN 博客，哈佛结构和冯·诺伊曼结构的区别，https://blog.csdn.net/fuhanghang/article/details/84064530